

SJTech Solutions AI Developer - What to Learn

Overview

This is the biggest discipline gap of any JD you've tailored for. Your AI experience (Echo, Vibe) is entirely in the LLM-application layer — calling pre-trained models via API, RAG, prompt engineering, agents. This JD wants classical ML/DL — training models yourself on data. These are genuinely different skill sets, and the JD lists none of the LLM/agent/RAG work you actually have as required. Be honest about this distinction in interviews rather than conflating "I use AI APIs" with "I train ML models."

MUST learn

Supervised & unsupervised learning — the core ML vocabulary

- What it is: supervised learning = training a model on labeled data (input → known correct output) — classification (spam/not-spam) and regression (predict a number) are the two main types. Unsupervised learning = finding structure in unlabeled data — clustering (grouping similar items) and dimensionality reduction are the main types.
- Why the JD wants it: named explicitly as required skills — this is the foundational vocabulary of the whole ML discipline.
- The 20%: work through ONE supervised example (e.g. logistic regression or a decision tree classifier on a toy dataset like Iris or Titanic using scikit-learn) and ONE unsupervised example (k-means clustering on the same kind of toy dataset). That hands-on pair covers the vocabulary interviewers will probe: train/test split, features/labels, overfitting, accuracy/precision/recall for supervised; clusters/centroids for unsupervised.
- Bridge: you already think in terms of inputs → transformations → outputs from building REST APIs and RAG pipelines — the new part is that the "transformation" here is learned from data rather than hand-coded logic.

Deep learning — neural networks (PyTorch or TensorFlow/Keras)

- What it is: deep learning uses neural networks (layers of weighted connections) to learn complex patterns — the technology underlying the LLMs you already use via API, but here you'd be building/training the networks yourself, not calling a finished one.
- Why the JD wants it: named explicitly, and it's the more advanced tier of the role's stated growth path.
- The 20%: build one tiny neural network (a basic image classifier on MNIST digits, the "hello world" of deep learning) in PyTorch or Keras — this single exercise covers layers, activation functions, loss functions, and the train loop (forward pass, backward pass, optimizer step) well enough to speak to the concepts.
- Bridge: you already understand that an LLM is a neural network trained on text (from using Echo's Gemini embeddings and Vibe's OpenAI calls) — the new part is the training loop itself, which you haven't had to touch since you consume pre-trained models via API.

Data Science tooling: pandas, NumPy, Jupyter

- What it is: pandas (tabular data manipulation — like a programmable spreadsheet), NumPy (fast numerical array operations, the math foundation under most ML libraries), Jupyter notebooks (interactive, cell-by-cell Python environment standard for data science work).
- Why the JD wants it: "Data Science" is a named required skill — these three tools are the daily-driver toolkit for it.
- The 20%: load a CSV with pandas, do basic exploration (`.describe()`, `.groupby()`, filtering/cleaning), and know NumPy arrays vs. Python lists (vectorized operations, no explicit loops). This is a different

daily workflow from your Node/TypeScript backend work, but the underlying logic (filter, transform, aggregate data) is familiar from working with API responses and analytics data (Mixpanel events).

GOOD to learn

Automation scripts in Python

- What it is: standalone Python scripts (not a full application) that automate a repetitive task — file processing, scheduled data pulls, batch operations.
- Why the JD wants it: named as a specific day-to-day responsibility, likely for internal tooling around the AI/ML pipeline.
- The 20%: this maps directly onto scripting you may already do informally — the shift is just doing it in Python syntax instead of JS/bash. Not a conceptual gap, just a language-familiarity one.

How to frame the gap honestly

Your genuine strength is the LLM-application layer (RAG, prompt engineering, agents, vector search) which this JD doesn't ask for at all — don't overstate it as equivalent to classical ML. In an interview, be direct: you have strong Python/CS fundamentals and applied GenAI experience, and you're building toward the classical ML/DL side this role needs, starting from the exercises above. That's a more credible story than implying deep ML experience you don't have.